

TRAFFIC_TIMER.CPP

```

/* Traffic timer.cpp */
#include "WProgram.h"
void Small_delay(void);
void Pulse_clock(void);
void Pulse_le(void);
void Switch2special_5916(void);
void Send_byte_5916 (unsigned char Display_byte);
void Active_output_5916(void);
void Inactive_output_5916(void);
void Refresh_grid(void);
void Set_pins_dir(void);
void Init_timer(void);
void Sense_input(void);
void Set_time(void);
void Binary2segment(unsigned int val);
void setup();
void loop();
void Reset_serial_packet(void);
void Act_one_tick_over();
void Act_one_sec_over();
void Act_half_sec_over(void);
void Send_packet(unsigned int x);
void Blankout_display();
void Act_setup();
unsigned char Grid_no, Seg_unit, Seg_ten, Seg_hun;
unsigned char I, J, K;
unsigned char Bit_5916, Small_tick_over,
One_tick_over, Half_sec_over, One_sec_over,
One_min_over;
unsigned char Red_input, Green_input, Current_red, Current_green, New_red_found, New_green_found;
unsigned char Ticks, Secs, Mins, New_red_count, New_green_count, Color;
unsigned char Numeric_table[10] = {0x0, 0xf8, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x98};
/* Bit 7 6 5 4 3 2 1 0
   G E D C F dp A B
*/
#define FND_BLANK 0x00
#define DP_ON 0x04
//unsigned char Numeric_table[10] = {0x0, 0xf8, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x98};
/*Bit 7 6 5 4 3 2 1 0
   dp g f e d c b a
*/
#define UNIT_G 12
#define UNIT_R 13
#define TEN_G 10
#define TEN_R 11
#define HUN_G 8
#define HUN_R 9
#define SCL 19
#define SDA 18
#define OE 17
#define LE 16
#define CLK 15
#define SDI 14
#define PI 7
#define P2 6
#define SDO 5
#define GREEN_SENSE 4
#define RED_SENSE 3
#define RE_LE 2
#define TXD 1
#define RXD 0
#define INACTIVE_GRID 0
#define ACTIVE_GRID 1
#define ACTIVE_INPUT 0
#define INACTIVE_INPUT 1
#define COLOR_RED 1
#define COLOR_GREEN 2
#define COLOR_ORANGE 3
#define COLOR_OFF 0
#define MAX_GRID_NO 2
/*This file is specific to TLC5916
All routines specific to TLC5916 will be here*/
void Small_delay(void)
{
  for (J=0; J<2; J++)
  {
    J++;
    J--;
  }
}

void Pulse_clock(void)
{
  digitalWrite(CLK,HIGH);
  Small_delay();
  digitalWrite(CLK,LOW);
  Small_delay();
}

void Pulse_le(void)
{
  digitalWrite(LE,LOW);
  Small_delay();
  digitalWrite(LE,HIGH);
  Small_delay();
  digitalWrite(LE,LOW);
  Small_delay();
}

void Switch2special_5916(void)
{
  digitalWrite(OE,HIGH);
  Small_delay();
  digitalWrite(LE,LOW);
  Small_delay();
  Pulse_clock();
  Small_delay();
  digitalWrite(OE,HIGH);
  Small_delay();
  Pulse_clock();
  Small_delay();
}

void Switch2normal_5916(void)
{
  digitalWrite(OE,HIGH);
  Small_delay();
  digitalWrite(LE,LOW);
  Small_delay();
  Pulse_clock();
  Small_delay();
  digitalWrite(OE,HIGH);
  Small_delay();
  Pulse_clock();
  Small_delay();
}

void Send_byte_5916 (unsigned char Display_byte)
{
  digitalWrite(OE,HIGH);
  Small_delay();
  digitalWrite(LE,LOW);
  Small_delay();
  Pulse_clock();
  Small_delay();
  digitalWrite(OE,HIGH);
  Small_delay();
  Pulse_clock();
  Small_delay();
  digitalWrite(LE,LOW);
  Small_delay();
  for (I=0; I<8; I++)
  {
    if (Display_byte & 0x80)
      digitalWrite(SDI,HIGH);
    else
      digitalWrite(SDI,LOW);
    Display_byte = Display_byte<<1;
    Small_delay();
    Pulse_clock();
    digitalWrite(SDI,LOW);
  }
  Small_delay();
  Pulse_le();
}

void Active_output_5916(void)
{
  Small_delay();
  digitalWrite(OE,LOW);
  Small_delay();
}

void Inactive_output_5916(void)
{
  Small_delay();
  digitalWrite(OE,HIGH);
  Small_delay();
}

void Refresh_grid(void)
{
  digitalWrite(UNIT_G,INACTIVE_GRID);
  digitalWrite(UNIT_R,INACTIVE_GRID);
  digitalWrite(TEN_G,INACTIVE_GRID);
  digitalWrite(TEN_R,INACTIVE_GRID);
  digitalWrite(HUN_G,INACTIVE_GRID);
  digitalWrite(HUN_R,INACTIVE_GRID);

  Grid_no++;
  if (Grid_no > MAX_GRID_NO)
    Grid_no = 0;
  if (Grid_no == 0)
    Send_byte_5916(Seg_unit);
  if (Grid_no == 1)
    Send_byte_5916(Seg_ten);
  if (Grid_no == 2)
    Send_byte_5916(Seg_hun);
  Active_output_5916();
  if (Grid_no==0)
  {
    if ((Color==COLOR_RED) || (Color == COLOR_ORANGE))
      digitalWrite(UNIT_R,ACTIVE_GRID);
    if ((Color==COLOR_GREEN) || (Color == COLOR_ORANGE))
      digitalWrite(UNIT_G,ACTIVE_GRID);
  }
  if (Grid_no==1)
  {
    if ((Color==COLOR_RED) || (Color == COLOR_ORANGE))
      digitalWrite(TEN_R,ACTIVE_GRID);
    if ((Color==COLOR_GREEN) || (Color == COLOR_ORANGE))
      digitalWrite(TEN_G,ACTIVE_GRID);
  }
  if (Grid_no==2)
  {
    if ((Color==COLOR_RED) || (Color == COLOR_ORANGE))
      digitalWrite(HUN_R,ACTIVE_GRID);
    if ((Color==COLOR_GREEN) || (Color == COLOR_ORANGE))
      digitalWrite(HUN_G,ACTIVE_GRID);
  }
}

#include <MsTimer2.h>
void Set_pins_dir(void)
{
  //All Output Pins set here
  pinMode(UNIT_G, OUTPUT);
  pinMode(UNIT_R, OUTPUT);
  pinMode(TEN_G, OUTPUT);
  pinMode(TEN_R, OUTPUT);
  pinMode(HUN_G, OUTPUT);
  pinMode(HUN_R, OUTPUT);
  pinMode(SCL, OUTPUT);
  pinMode(SDA, OUTPUT);
  pinMode(OE, OUTPUT);
  pinMode(LE, OUTPUT);
  pinMode(CLK, OUTPUT);
  pinMode(SDI, OUTPUT);
  pinMode(RE_LE, OUTPUT);
  pinMode(TXD, OUTPUT);

  //All Input Pins set here
  pinMode(SDO, INPUT);
  pinMode(P1, INPUT);
  pinMode(P2, INPUT);
  pinMode(GREEN_SENSE, INPUT);
  pinMode(RED_SENSE, INPUT);
  pinMode(RXD, INPUT);
}

void Init_timer(void)
{
  // set timer int at 5 msec
  MsTimer2::set(5,Set_time); // 5ms period
  MsTimer2::start();
}

void Sense_input(void)
{
  Current_red = digitalRead(RED_SENSE);
  Current_green = digitalRead(GREEN_SENSE);

  if (Current_red != Red_input)
  {
    if (New_red_count<10)
    {
      New_red_count++;
    }
  }
}

```